

Design and Implementation of Cache Memory using Verilog HDL

Akshay Srikrishnan

Manipal Institute of Technology, Bengaluru

Tool: Xilinx Vivado

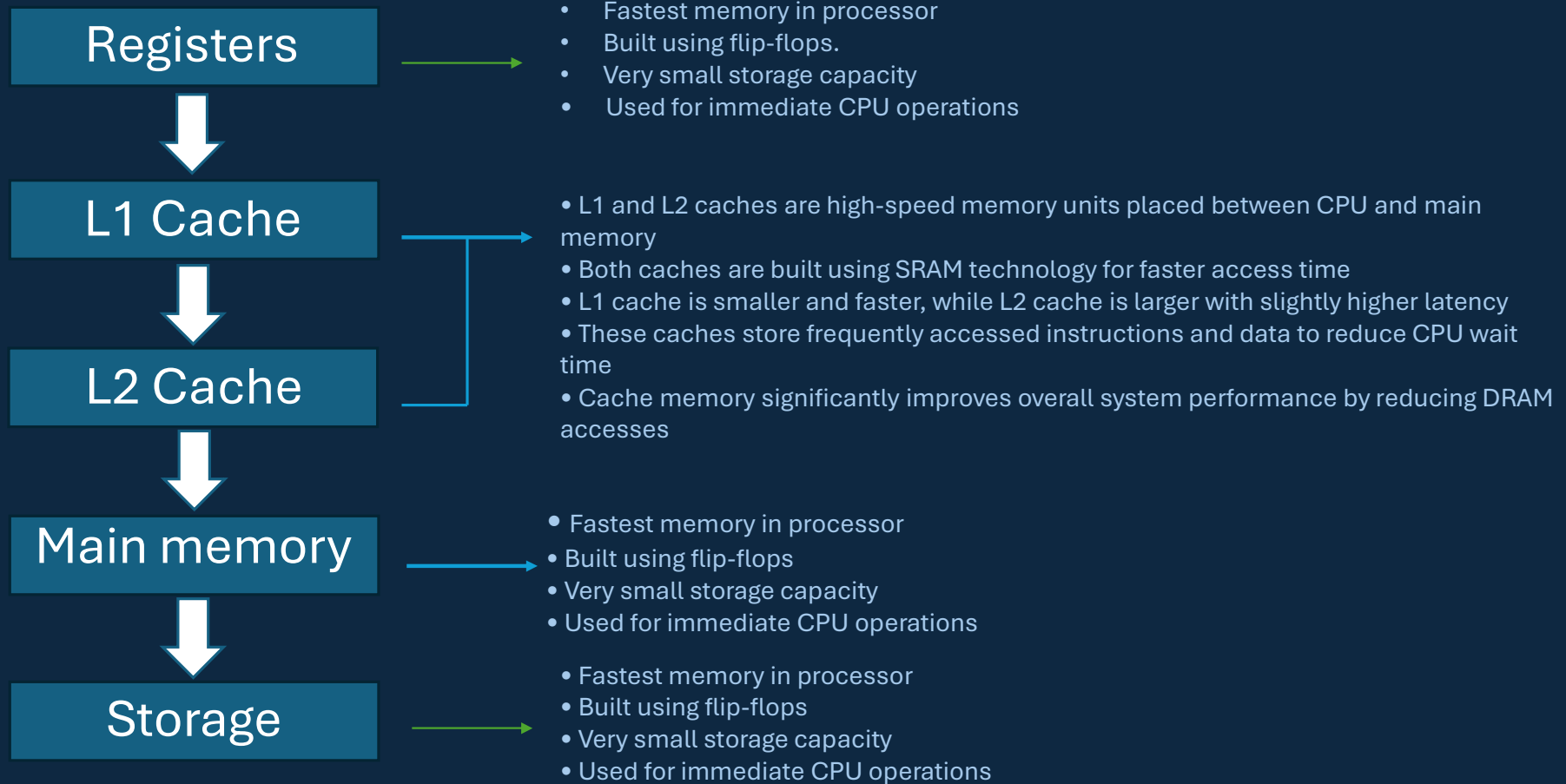
|

Language: Verilog HDL

|

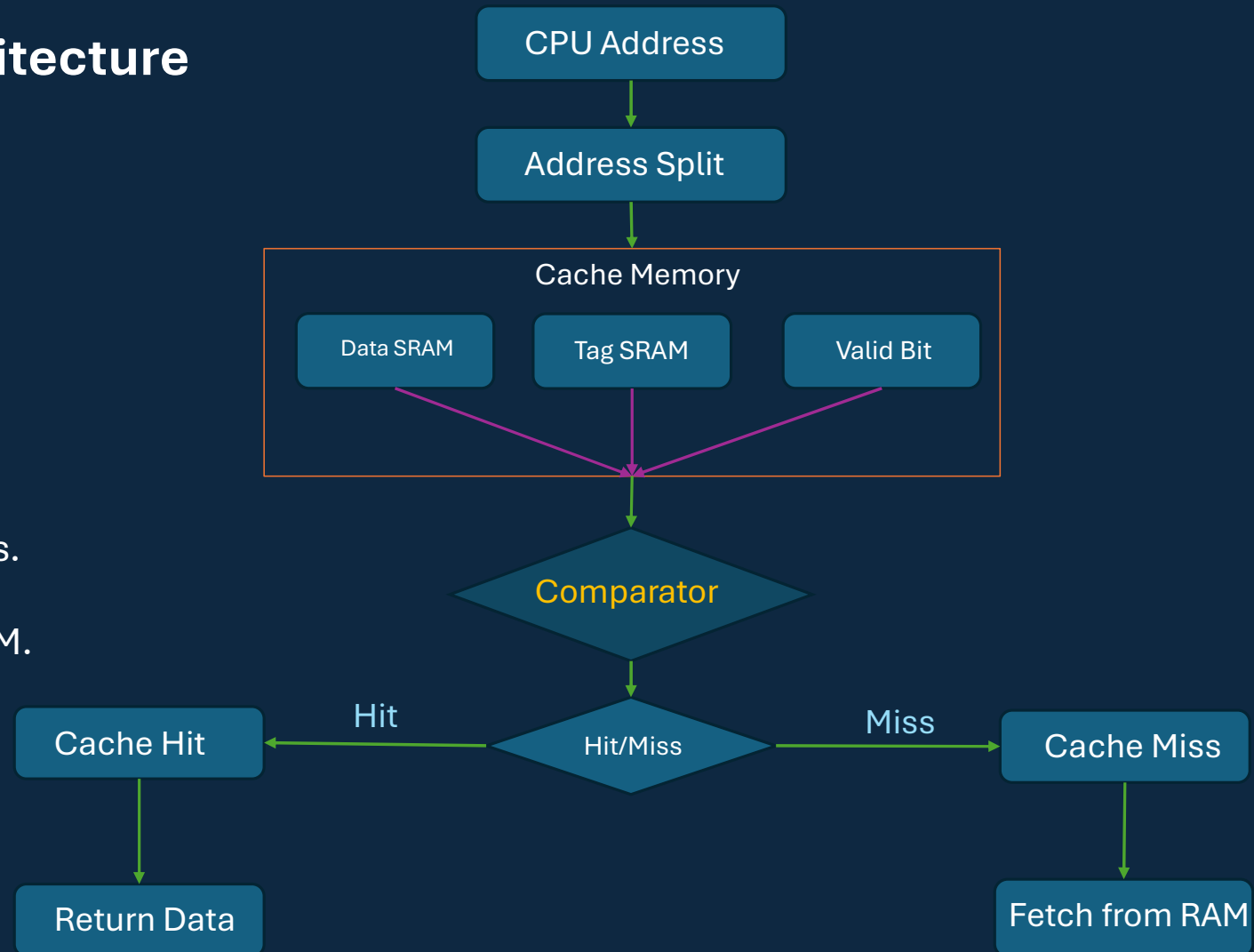
May 2026

Memory Hierarchy



Cache Memory Architecture

- Address Split → Tag + Index.
- SRAM stores cache data.
- Comparator performs hit/miss detection.
- Cache Hit → Fast data access.
- Cache Miss → Fetch from RAM.



Data SRAM

- 4×4 SRAM contains 4 memory locations
- Each location stores 4-bit data
- Total storage capacity = 16 bits
- Address lines select one memory row
- SRAM performs fast read/write operations
- Used as Data SRAM in cache memory

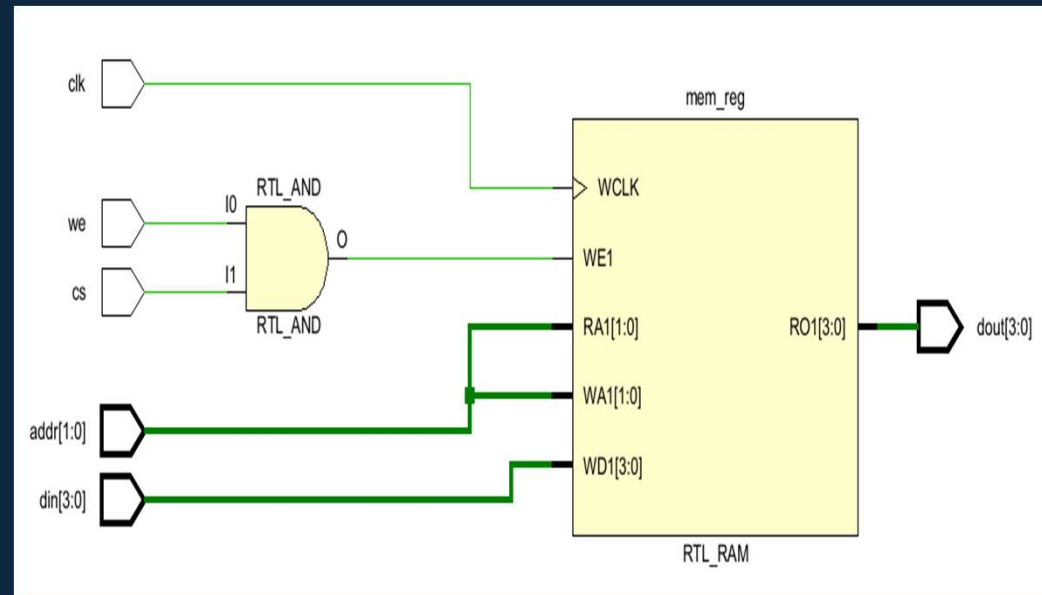
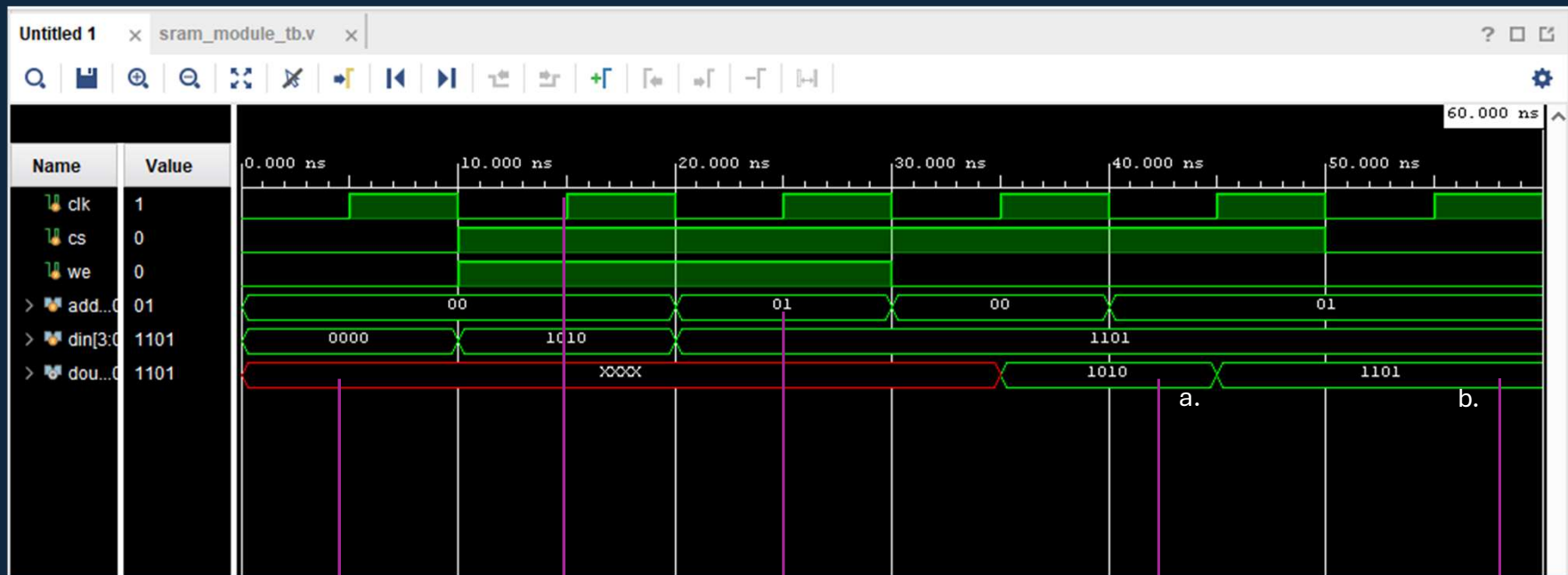


Fig 1. Design of Data SRAM

Simulation of 4X4 SRAM (Data SRAM)



Initial Unknown
state(Uninitialized
SRAM Contents)

Addr=00
din=1010
we=1
Write operation: Data
1010 stored into mem
location 00

Addr=01
din=1101
we=1
Write operation: Data
1101 stored into mem
location 01

Read operation:
a) Stored data retrieved
from address 00.
b) Stored data retrieved
from address 01

Tag SRAM

- Stores cache tag information.
- Contains 4 memory locations.
- Each location stores 3-bit tag.
- Used for cache hit detection.
- Tag compared with CPU address tag.
- Works together with comparator logic.

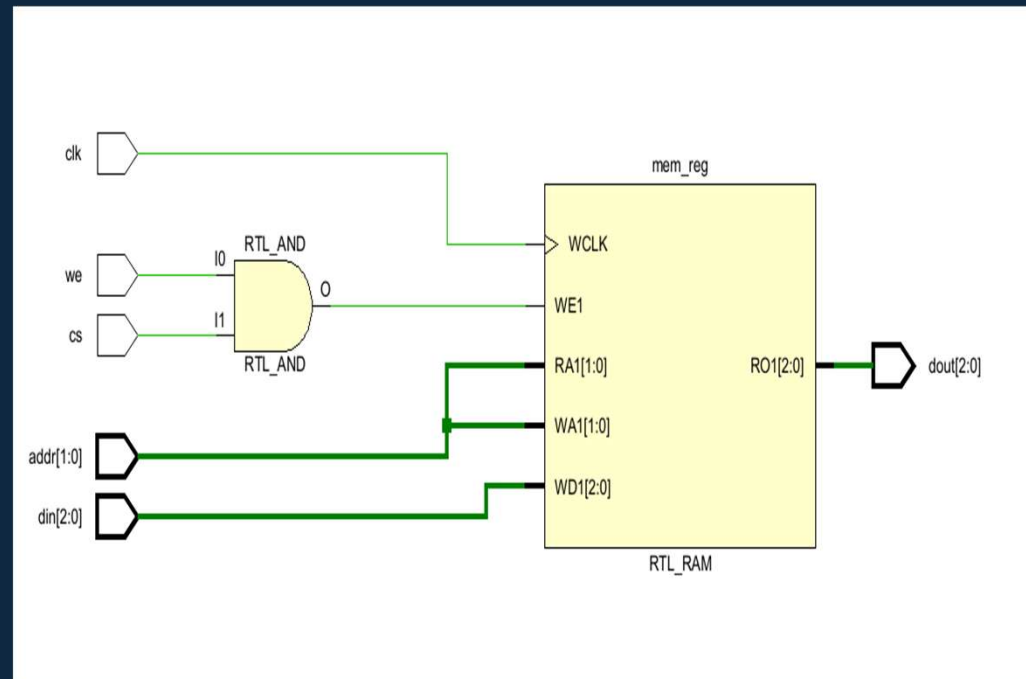
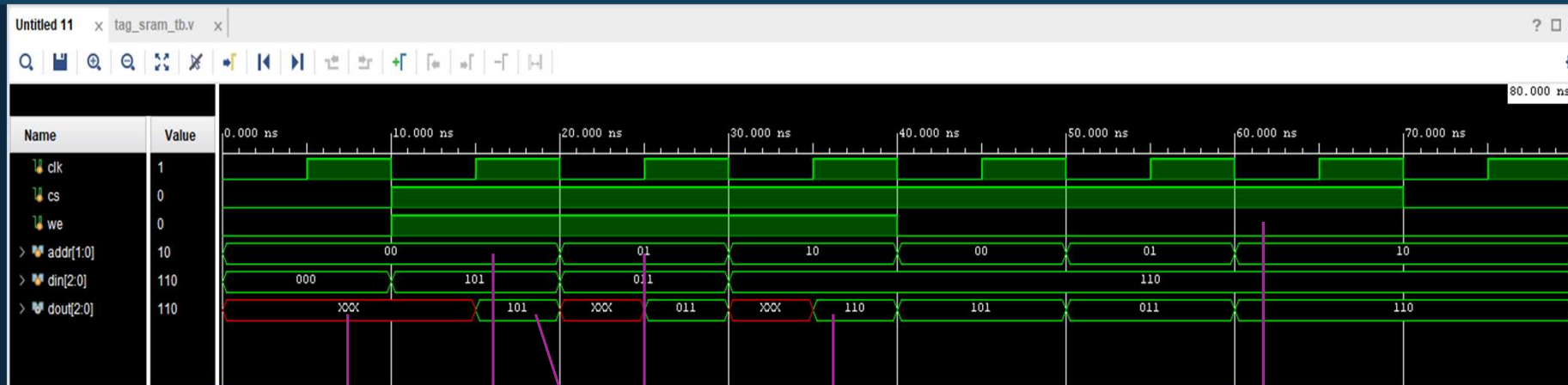


Fig 1. Design of Tag SRAM

Simulation of 4X3 SRAM (Tag SRAM)



WRITE

Initial unknown tag values before write operation.

WRITE

Tag value 101 written into cache line 00.

READ

Stored Tag 101 retrieved for comparison.

WRITE

Tag value 011 written into cache line 01.

READ

Tag value 110 written into cache line 10.

➤ Tag successfully read for hit/miss detection.

WE = 1 → Tag write operation.

WE = 0 → Tag read operation.

READ

We:0
Stored tag 110 retrieved for hit/miss detection.

Comparator/Hit Logic

- Compares CPU tag with stored tag.
- Valid bit checked before comparison.
- Matching tags generate cache hit.
- Mismatched tags generate cache miss.
- Hit signal enables cache data output.
- Core block for cache decision making.

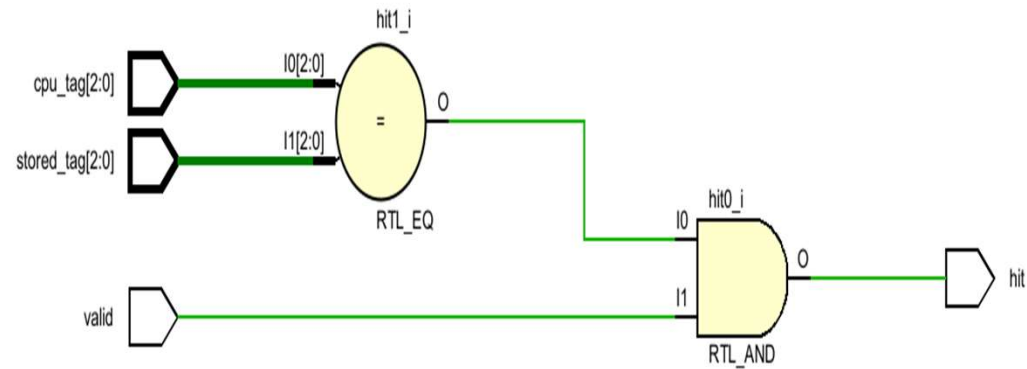
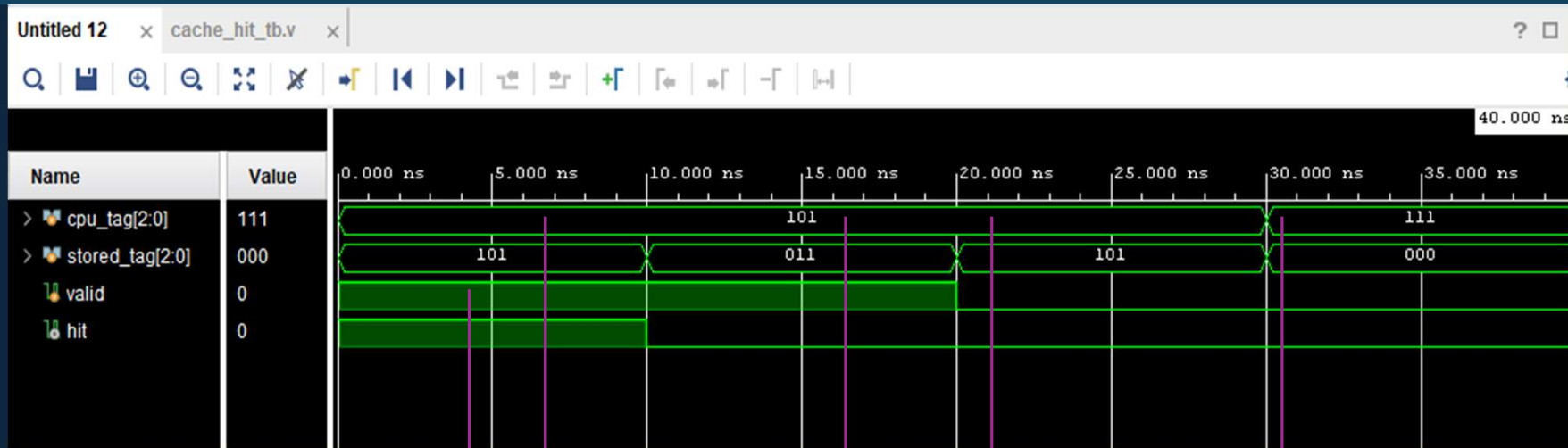


Fig 1. Design of Comparator/Hit Logic

Simulation of Comparator for Hit or Miss



HIT:
Matching tag values with
valid bit enabled
generate cache hit.

MISS:
Different tag values
result in cache miss.

Valid bit must be
enabled for cache hit
generation.

Comparator
successfully detects
matching cache line.

Cache miss generated
due to tag mismatch.

Comparator checks whether:

CPU Tag = Stored Tag

If tags match and valid bit = 1,
cache hit is generated.

Valid Bit Array + Address Split Logic

- Address Split Logic divides the CPU address into tag bits and index bits
- Tag bits are used for cache comparison operations
- Index bits select the required cache memory line
- Valid Bit Array stores validity information for each cache line
- Valid bit = 1 indicates valid cache data is present
- Valid bit = 0 indicates cache line is empty or invalid
- Comparator checks both tag match and valid bit status
- Together, these modules enable accurate cache hit/miss detection

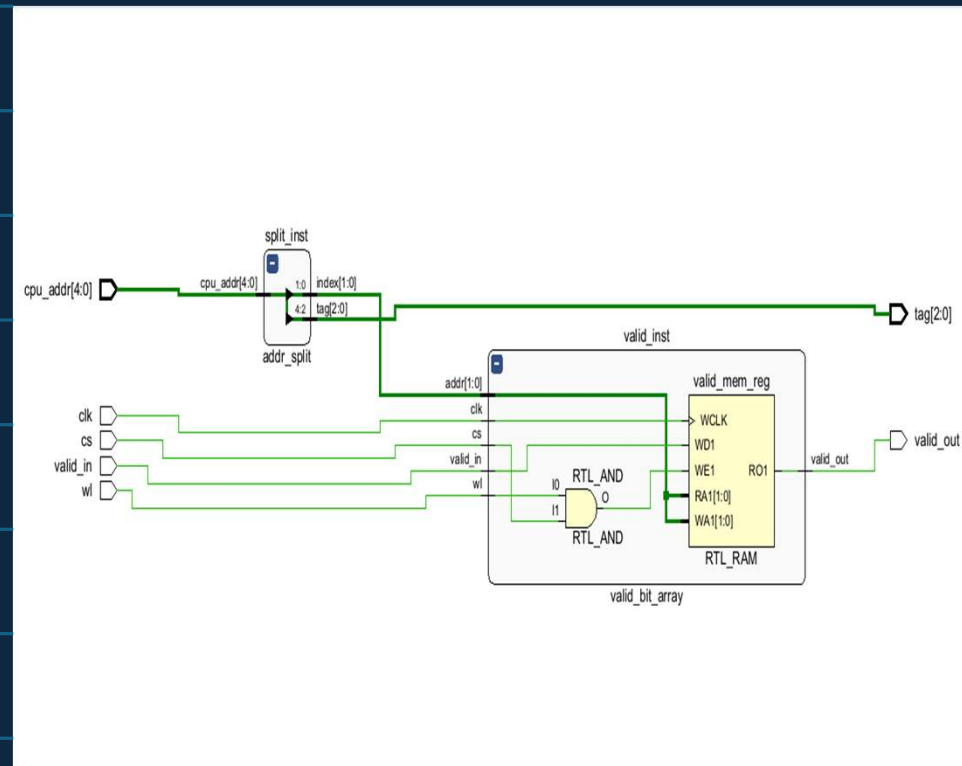
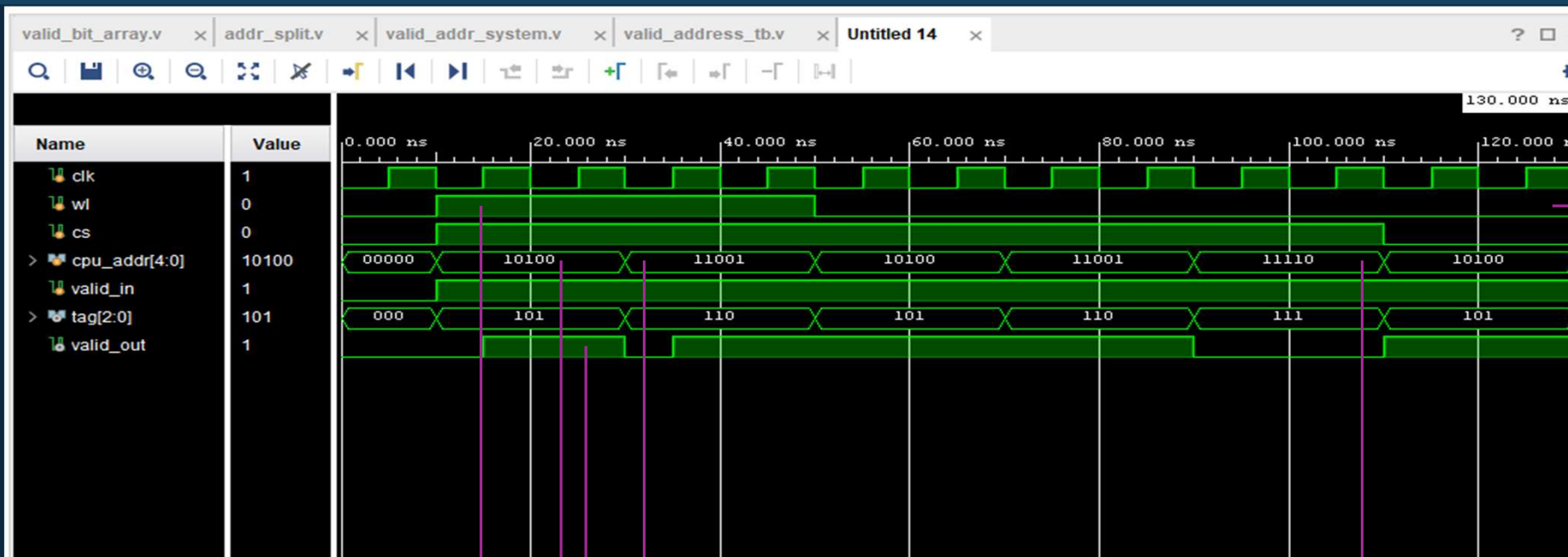


Fig 1. Valid Bit Array+Address Split Logic

Simulation of Valid Bit array+Address split logic



WL = 1 → Valid bit write operation
WL = 0 → Valid bit read operation

CPU address split into:

- a) TAG = 101
- b) INDEX = 00

WL = 1
valid_in = 1

Valid bit written into selected cache line

Address split logic extracts tag and index bits

valid_out = 1

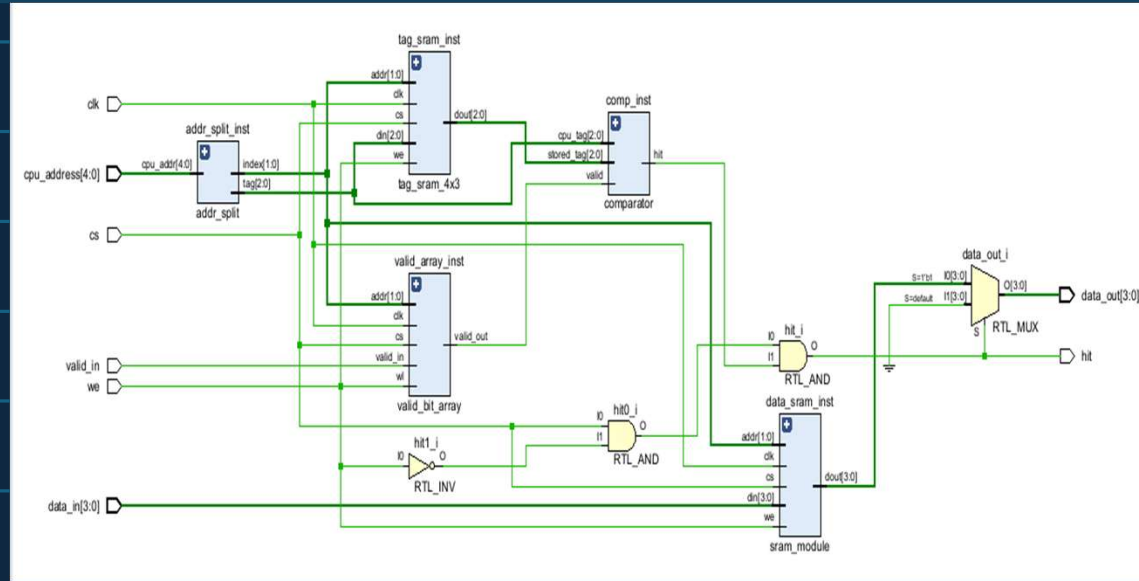
Selected cache line contains valid data

Invalid cache line detected
(valid bit = 0)

Address split logic divides the CPU address into tag and index bits.
The valid bit array verifies whether the selected cache line contains usable cache data.

Cache Top Module

- Integrates all cache datapath modules.
- Connects SRAM, tag and valid arrays.
- Performs cache hit/miss detection.
- Controls cache read/write operations.
- Outputs final cache data to CPU.
- Represents complete cache datapath.



Cache Top Flow:

CPU Address

Address Split

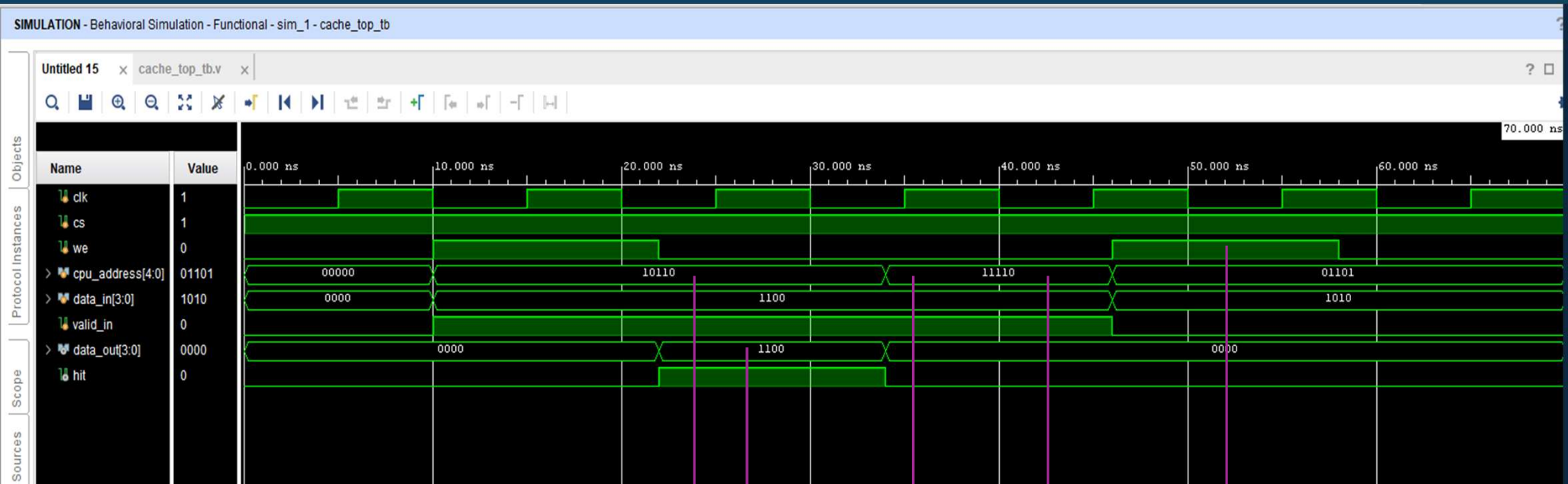
Cache Mem
Access

Comparator

H/M Logic

Cache Output

Simulation of Cache Top Module



The cache_top module integrates **address split logic**, **SRAM arrays**, **valid bit array**, and **comparator logic** to perform complete **cache hit/miss detection** and data retrieval operations.

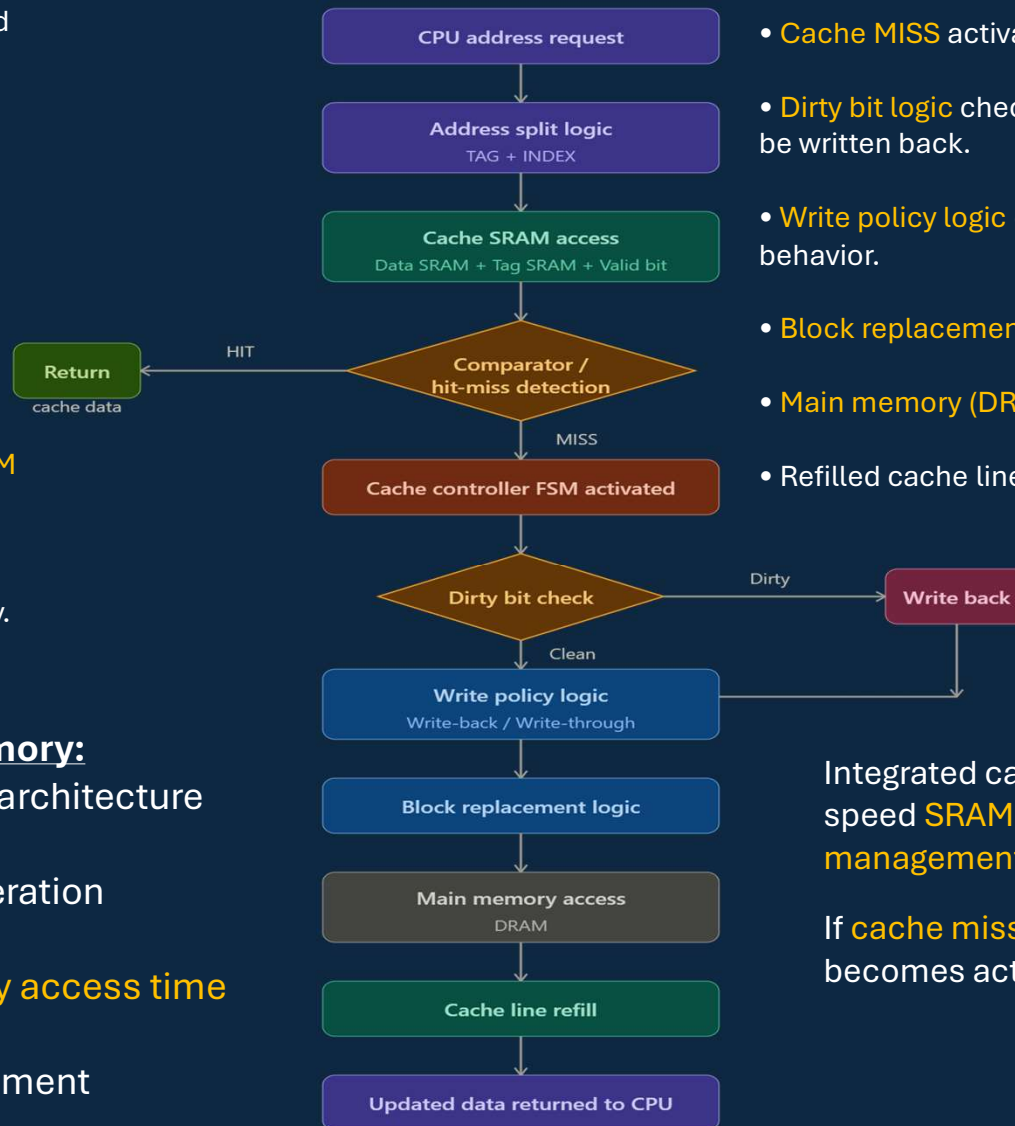
Final Cache System

- cache_top performs primary cache lookup operations
- Address split logic generates tag and index bits
- SRAM arrays store cache data, tags, and validity information
- Comparator logic determines cache hit or cache miss
- Cache controller FSM coordinates miss handling operations
- Dirty bit array tracks modified cache lines
- Write policy logic controls memory update behavior
- Block replacement logic selects cache line replacement during misses
- Main memory module supplies data during cache refill operations
- All modules operate together as a unified cache memory subsystem

- CPU address is divided into **TAG** and **INDEX** bits.
- **Data SRAM**, **Tag SRAM** and **Valid Bit Array** are accessed simultaneously.
- **Comparator** checks tag matching and valid bit status.
- **Matching tag** and **valid bit** generate cache HIT.
- **Cache HIT** enables **immediate SRAM** data return to CPU.
- **SRAM-based cache** access significantly reduces memory latency.

Advantages of Cache Memory:

- **High-speed** SRAM based architecture
- **Parallel cache** lookup operation
- **Reduced average memory access time**
- Efficient **hit/miss** management

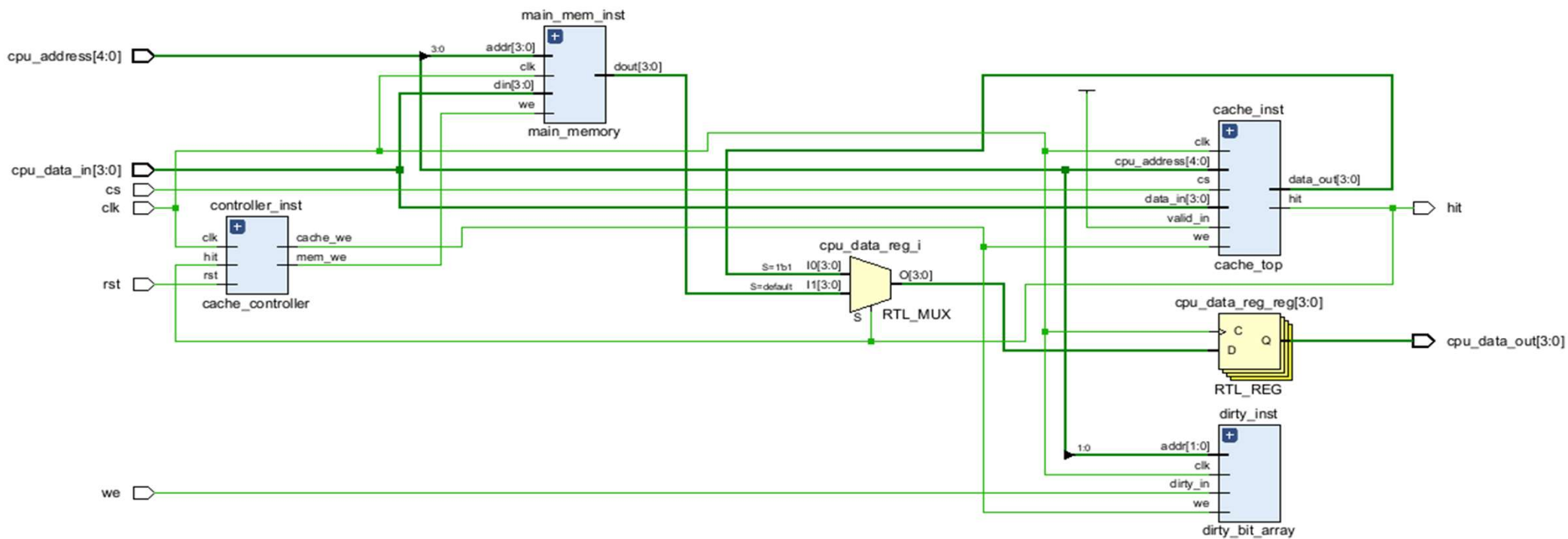


- **Cache MISS** activates the cache controller FSM.
- **Dirty bit logic** checks whether modified cache data must be written back.
- **Write policy logic** controls memory synchronization behavior.
- **Block replacement** logic selects cache line replacement.
- **Main memory (DRAM)** supplies updated cache data.
- Refilled cache line is returned back to **CPU**.

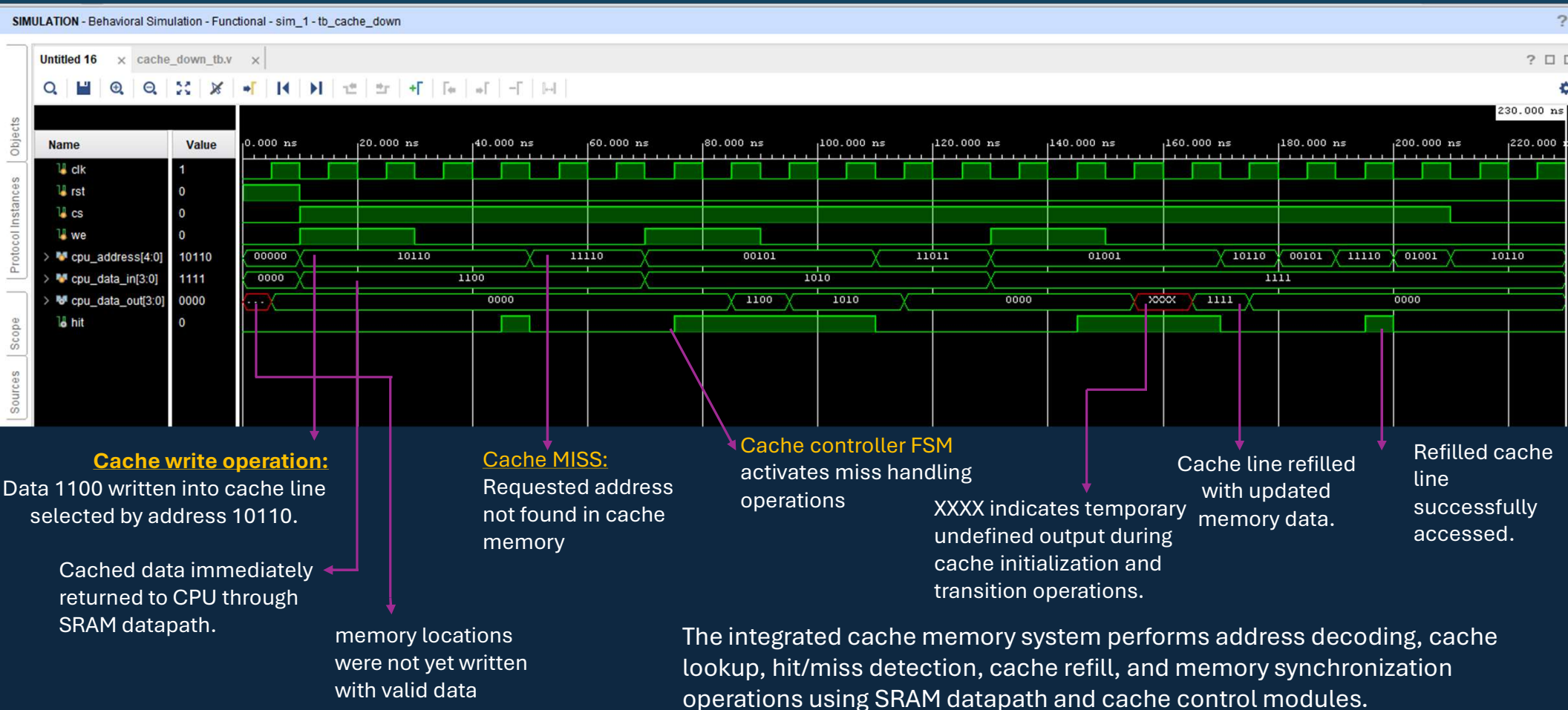
Integrated cache architecture combining high-speed **SRAM datapath** and **memory management control logic**.

If **cache miss** occurs, the control subsystem becomes active.

Schematic of Cache Memory



Simulation of Full Cache Memory:



Conclusion:

- Successfully designed and simulated a complete direct-mapped cache memory architecture using Verilog HDL.
- Implemented SRAM-based cache storage, tag comparison, valid bit logic, and cache hit/miss detection.
- Integrated cache controller FSM, dirty bit logic, write policies, replacement logic, and main memory interface.
- Verified cache read/write operations, cache hit handling, and cache miss refill operations through behavioral simulation.
- Demonstrated practical implementation of memory hierarchy and cache management concepts.